COMP5310: Principles of Data Science

W9: Linear Regression & Logistic Regression

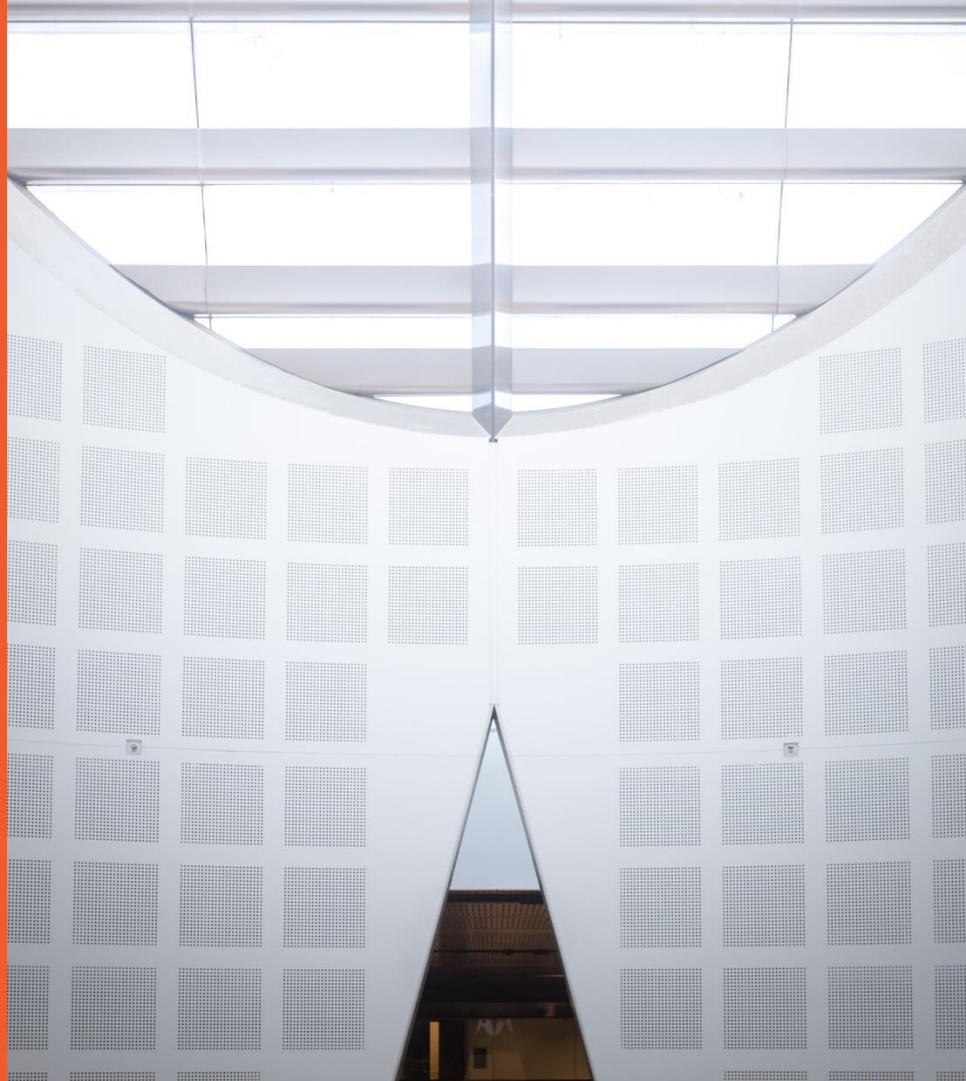
Presented by

A/Prof. Nguyen Tran

School of Computer Science

Based on slides by previous lecturers of this unit of study

THE UNIVERSITY OF SYDNEY



Last week: Clustering and Dimensionality Reduction

Objective

Learn techniques for unsupervised learning, with tools in Python.

Lecture

- Clustering algorithms
- Evaluating clustering
- Choosing k
- Principal Component Analysis

Readings

- Intro to Data Mining, Ch. 7 https://www-users.cse.umn.edu/~kumar001/dmbook/ch7_clustering.pdf
- Data Science from Scratch, Ch. 20

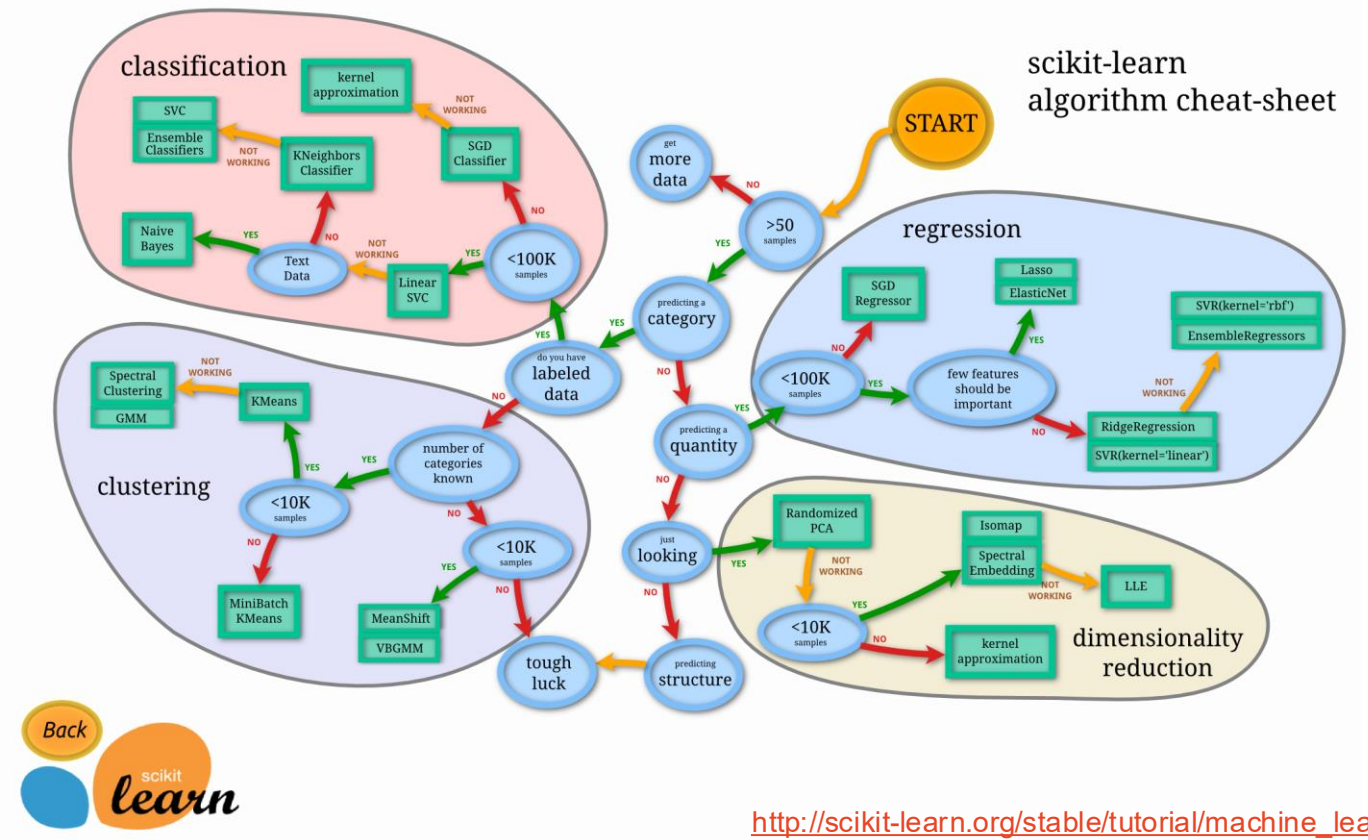
Exercises

- sklearn: clustering and PCA

Supervised Learning:

- We'll now focus on supervised machine learning techniques
  - **Simple linear regression**
  - **Multiple linear regression**
  - **Logistic regression**
  - Decision tree
  - Naïve Bayes

Machine learning map from scikit-learn



http://scikit-learn.org/stable/tutorial/machine_learning_map/

Predictive Modelling

Refund	Status	Income	Cheat
Yes	Single	125K	No
No	Married	100K	No
No	Single	70K	No
Yes	Married	120K	No
No	Divorced	95K	Yes
No	Single	85K	Yes
Yes	Single	90K	Yes

Refund	Status	Income	Cheat
No	Married	80K	?

Classification

Carbon level(%)	Purity (%)
0.99	90.01
1.02	86.05
1.15	91.43
1.29	93.74
1.46	96.73
1.36	94.45
0.87	87.59

Carbon level(%)	Purity (%)
1.32	?

Regression

- Many data science problems involve predicting an outcome from available data.

$$X \rightarrow \hat{Y}$$

- A predictive model learns a function

$$\hat{Y} = f(X)$$

- X: Input features
- Y : actual target value
- $\hat{Y}$: predicted target value
- If Y is numeric, the task is regression
- If Y is categoric, the task is classification

From predictive modelling to regression

- In this lecture, we will study different forms of the predictive function $f(X)$

Topic	Model Form	Output
Simple linear regression	$\hat{y} = \alpha + \beta x$	Numeric value
Multiple linear regression	$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_1 x_1 + .. + \theta_d x_d$	Numeric value
Gradient descent	Optimisation method for learning parameters	Updated model parameters
Logistic regression	$\hat{p} = P(Y = 1 \mid X)$	Probability/Class label

How Predictive Models are built

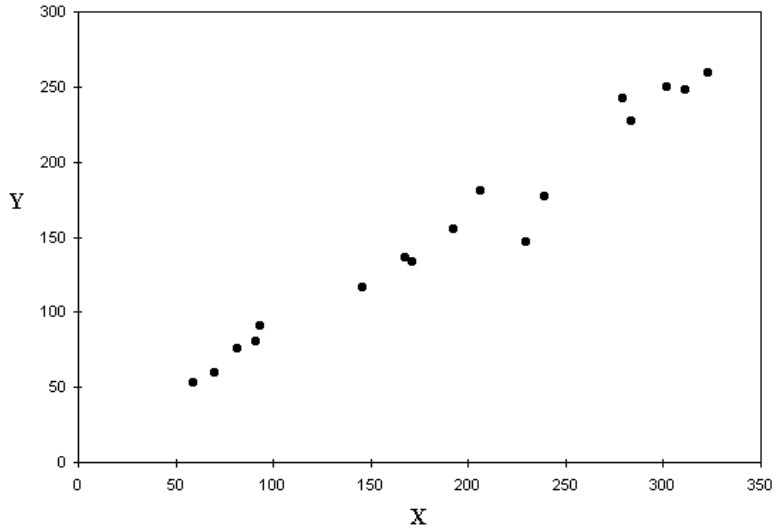
– Most supervised predictive models involve three key components:

Component	Key question	In this lecture
Model Form	What does $f(X)$ look like?	Simple linear regression, multiple linear regression, logistic regression
Learning method	How do we find the parameters?	cost functions (Least squares, cross-entropy), gradient descent
Evaluation metric	How good are the predictions?	MSE, RMSE, R^2 ,classification metrics

Simple Linear Regression

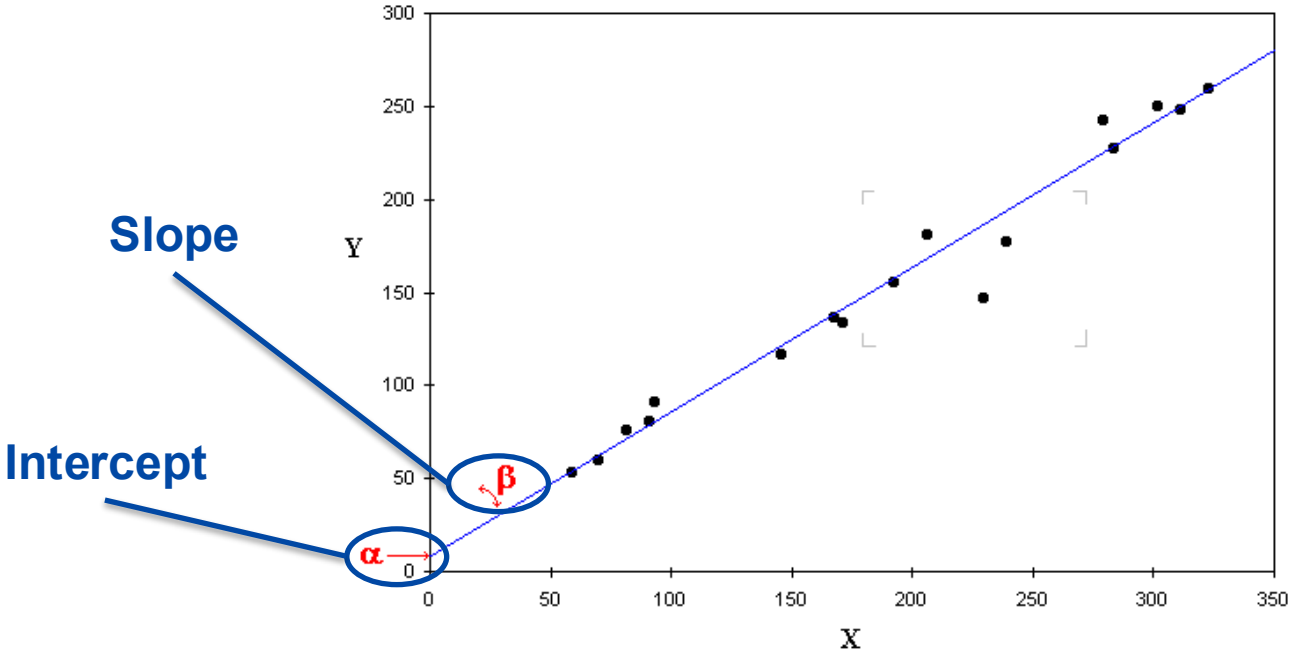
THE UNIVERSITY OF SYDNEY

Simple linear regression



- Method for finding the **line of best fit** between the dependent variable Y and the independent variable X
- **Simple**: only one independent variable

What's the line that best predicts Y from X?



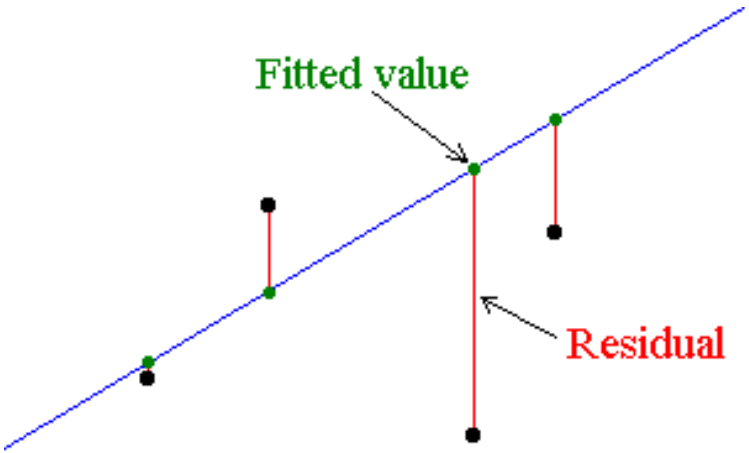
$$\hat{y}_i = \hat{\alpha} + \hat{\beta} x_i$$

Fitting SLR: learn α and β

$$y_i = \alpha + \beta x_i + \varepsilon$$

- **α**: Intercept (where the line crosses the y axis)
- **β**: Slope (direction and steepness of the line)
- **ε_i**: Error (error term describing variation of data)
- **Y_i** is the dependent variable (response).
- **X_i** is the independent variable (predictor).

Fitting SLR: Minimize sum of squared errors



- **Error/residual:** difference between the observed value and predicted value

  $(y_{actual} - y_{predicted})$

- Sum of squared errors:

$$\varepsilon = SSE = \sum (y_i - \hat{y}_i)^2$$

Sum Error Square

http://home.ku.edu.tr/yihlamur/public_html/Bitirme%20Projesi%20-%20Ger%C3%A7ek%20Data%20D%C3%BCzenlenmi%C5%9F/regression/minitab%20regresion%20hakk%C4%B1nda%20g%C3%BCzel%20bilgiler.htm

Least Squares Models

- Many prediction models make predictions of the form:
  - $\hat{y}_i = f(x_i)$
  - Simple linear regression is a special case of the general form
- A least squares model chooses model parameters to make the squared residuals (errors) as small as possible
  - $\min \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$
- Ordinary Least Squares (OLS) is the standard least squares method for linear regression:
  - The model is linear in its parameters
  - All observations/data points are treated equally

Ordinary Least Squares (OLS)

- Our goal is to find the optimal value of $\hat{\alpha}$ and $\hat{\beta}$ such that $\frac{1}{n}\sum_{i=1}^{n}\left(\left(\hat{\alpha}+\hat{\beta}x_i\right)-y_i\right)^2$ is the minimum
  - Mean Square Error
- Using calculus

$$\hat{\beta}=\frac{\sum\left(x_i-\bar{x}\right)\left(y_i-\bar{y}\right)}{\sum\left(x_i-\bar{x}\right)^2}=\frac{\mathrm{cov}(x,y)}{\mathrm{Var}(x)}=\frac{r(x,y)*sd(y)}{sd(x)}$$

$$\hat{\alpha}=\bar{y}-\hat{\beta}\bar{x}$$

sd is the standard deviation and ***r*** is the correlation

$\bar{x}=\frac{1}{n}\sum_{i=1}^{n}x_i$ is the average of x_i values

$\bar{y}=\frac{1}{n}\sum_{i=1}^{n}y_i$ is the average of y_i values

Training and Testing Data

- Fitting a model and evaluating a model are different tasks
- For predictive modelling, it is typical to split the data into training/test set
- A model may fit the training data well but perform poorly on new data

Dataset	Purpose
Training data	Used to learn the model parameters
Test data	Kept aside to check how well the model performs on unseen data

Evaluation Metrics

- Can we use Mean Squared Error (MSE) to measure how well the model fits the data?
  - Yes. OLS directly minimises MSE on the training data.
- What is the limitation of MSE?
  - MSE is measured in squared units.
  - This makes it difficult to interpret in the original context of the problem.
- A more interpretable metric: RMSE
  - RMSE stands for Root Mean Squared Error.
  - RMSE is the square root of MSE.
  - RMSE is measured in the same unit as the response variable y.
- **However, both MSE and RMSE are scale-dependent**
  - Their values depend on the scale and range of y.
  - For example, predicting exam marks out of 100 and predicting annual income in dollars may produce very different MSE/RMSE values, even if the models have similar relative performance.

Example: Predicting exam mark from study hours

Mark = 50 + 5 * study hours

y = 50 + 5x

Student	Study hours (x)	Actual mark (y)	Predicted mark($\hat{y}$)	Error ($y - \hat{y}$)	Squared error
A	2	58	60	-2	4
B	4	73	70	3	9
C	6	76	80	-4	16
D	8	93	90	3	9

$$MSE = \frac{(4 + 9 + 16 + 9)}{4} = 9.5$$

$$RMSE = \sqrt{9.5} \approx 3.08$$

Based on RMSE, we can say that the prediction is typically off by 3 points

MSE and RMSE measure the size of prediction errors, they are useful but scale dependent

This might be good enough for score between 0~100, but it is very bad if the prediction is for satisfaction rating between 1~5.

Coefficient of determination (R^2)

- **R^2**: ratio of **explained variation in y** to **total variation in y**

$$R^2 = \frac{\sum(\hat{y}_i - \bar{y})^2}{\sum(y_i - \bar{y})^2} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} = 1 - \frac{SSE}{SST}$$

Sum of squared total

- Ranges from 0 to 1, measures goodness of fit, with higher values indicating better fit for the **linear** model
  - $R^2 = 0$: the model explains none of the variation in y
  - $R^2 = 1$: the model explains all variation in y
  - $R^2 = 0.75$: the model explains 75% of the variation in y

Example: Predicting exam mark from study hours

Student	Study hours (x)	Actual mark (y)	Predicted mark($\hat{y}$)	Error ($y - \hat{y}$)	Squared error
A	2	58	60	-2	4
B	4	73	70	3	9
C	6	76	80	-4	16
D	8	93	90	3	9

$$SSE = 4 + 9 + 16 + 9 = 38$$

$$\bar{y} = \frac{58 + 73 + 76 + 93}{4} = 75$$

$$SST = (58 - 75)^2 + (73 - 75)^2 + (76 - 75)^2 + (93 - 75)^2 = 618$$

$$R^2 = 1 - \frac{38}{618} = 0.94$$

Test your understanding

– A model predicting exam marks out of 100 has: $RMSE = 4$

Which interpretation is most appropriate?

A. The model explains 4% of the variation in exam marks.

B. The model's predictions are typically about 4 marks away from the actual marks.

C. The model has exactly 4 incorrect predictions.

D. The model's average squared error is 4 marks.

Test your understanding

– Two regression models have the same RMSE value of 3. Model A predicts exam marks out of 100. Model B predicts a satisfaction rating from 1 to 5. Which statement is most appropriate?

A. The two models have equally good predictive performance.

B. Model A is definitely worse because exam marks have a larger range.

C. RMSE = 3 may be acceptable for exam marks but very poor for a 1–5 rating.

D. The two models have equally poor predictive performance

Test your understanding

– A regression model has $R^2 = 0.72$, which statement is the best interpretation?

A. The model predicts 72% of the data points exactly.

B. The model has an average prediction error of 0.72.

C. The model explains about 72% of the variation in the response variable.

D. The model's slope is 0.72.

Multiple Linear Regression

THE UNIVERSITY OF SYDNEY

Multiple linear regression

$$Y = \alpha + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_d X_d$$

- Explain the relationship between:
  - **two or more** independent variables and
  - one dependent variable

Learning the Regression Parameters

- We use a linear prediction model:

$$\hat{y}^{(i)} = h_\theta(x^{(i)}) = \theta_0 + \theta_1 x_1^{(i)} + \cdots + \theta_d x_d^{(i)} = \theta^T x^{(i)}$$

$$\text{Assume } x_0^{(i)} = 1, \text{ so } \theta_0 \text{ is the intercept.}$$

- We choose θ to minimise the squared prediction errors

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

Note normal MSE would have a denominator of n

- For Ordinary Least Squares linear regression, this has a closed-form solution.

Machine learning in scikit-learn

```
from sklearn.linear_model import LinearRegression
lm = LinearRegression()
lm.fit(X_train, Y_train)
Y_test = lm.predict(X_test)

# We use the score method to get r-squared
print('R-squared:', lm.score(X_train, Y_train))
```

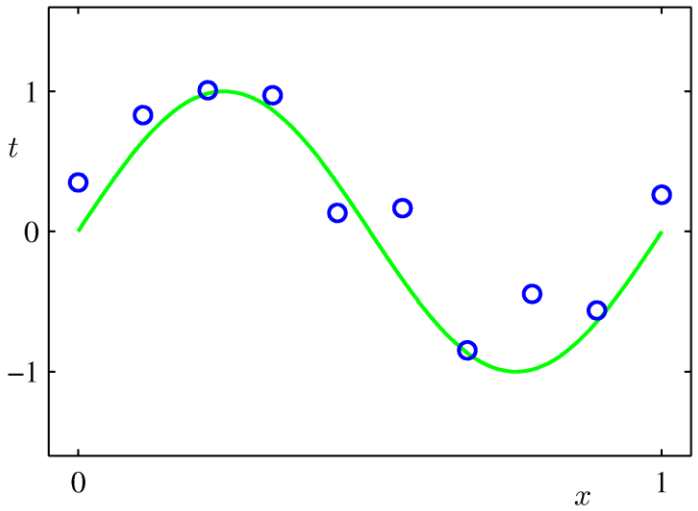
- **Fit(X, y)**: fits a model to the training data X, y
  - **X**: feature vectors
  - **y**: value
- **Predict(T)**: predict value for new data T

Model complexity and overfitting

THE UNIVERSITY OF SYDNEY

Increasing model flexibility

- A linear model an be extended by Polynomial transformed features
  - $\hat{y} = h_{\boldsymbol{\theta}}(\mathrm{x}) = \theta_0 + \theta_1 \mathrm{x}^1 + \theta_2 \mathrm{x}^2 + \ldots + \theta_d \mathrm{x}^\mathrm{d}$
- Although the curve is nonlinear in x, the model is still linear in the parameters: $\theta_0, \theta_1, \ldots, \theta_d$
- This allows the use of linear regression techniques to fit non-linear datasets.

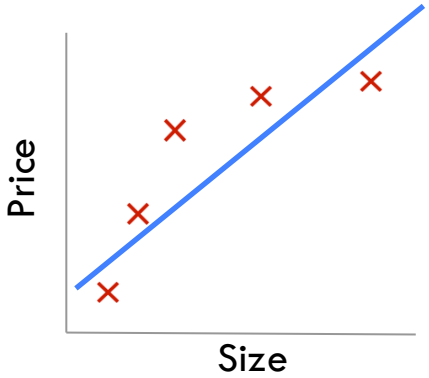


Model Complexity and fit

- A more flexible model can capture more complex patterns.
- But too much flexibility can fit noise rather than the underlying trend.

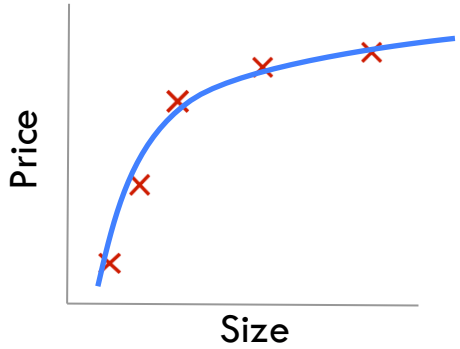
Model	Training fit	Test performance
Underfitting	Poor	Poor
Good fit	Good	Good
Overfitting	Very good	Often poor

Quality of fit example



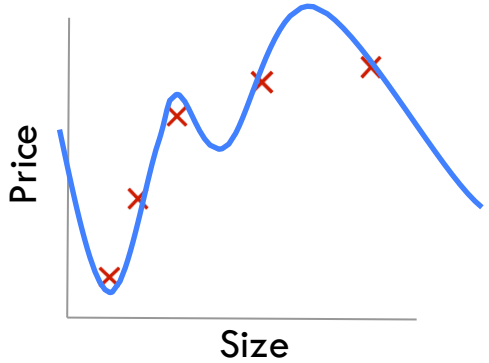
$\theta_0 + \theta_1 x$

Under-fitting (high bias)



$\theta_0 + \theta_1 x + \theta_2 x^2$

Correct fit



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

Overfitting (high variance)

How can we reduce overfitting?

- Use a simpler model
- Use fewer or more relevant features
- Collect more training data, if possible
- Use train/test split or cross-validation to check generalisation.
- Use regularization to discourage overly complex models.

What is regularization

- Ordinary training objective:
  - $cost = prediction\ error$
- Regularized training objective:
  - $cost = prediction\ error + penalty\ for\ complexity$
- For linear models, complexity is often measured using coefficient size
  - $large\ coefficients \Rightarrow more\ sensitive\ model$

Common types of regularization

- L2 and L1 regularization

Method	Penalty	interpretation
L1/Lasso	$\lambda \sum_{j=1}^{d} \lvert\theta_j\rvert$	penalises the sum of absolute coefficient values. It can shrink some coefficients exactly to zero, so it can act like automatic feature selection.
L2/Ridge	$\lambda \sum_{j=1}^{d} {\theta_j}^2$	Minimizes coefficients; pulls coefficients toward 0

λ controls regularization strength

Large λ: strong regularization

The intercept θ_0 is often not regularized

Gradient Descent

THE UNIVERSITY OF SYDNEY

How to learn θ

- For many other models/cost functions, there is no closed-form solution of θ

- A general search procedure for learning θ
  - Choose an initial value for $\boldsymbol{\theta}$
  - Iteratively choose a new value for $\boldsymbol{\theta}$ to reduce $J(\boldsymbol{\theta})$ (e.g., through gradient descent)
    - Until we reach a minimum

Intuition behind cost function (1 of 4)

We will use the simple linear regression function to help understand the cost function

Common form: $\hat{y} = \alpha + \beta x$

Generic form: $\hat{y} = h_\theta(x) = \theta_0 + \theta_1 x$

Cost function:

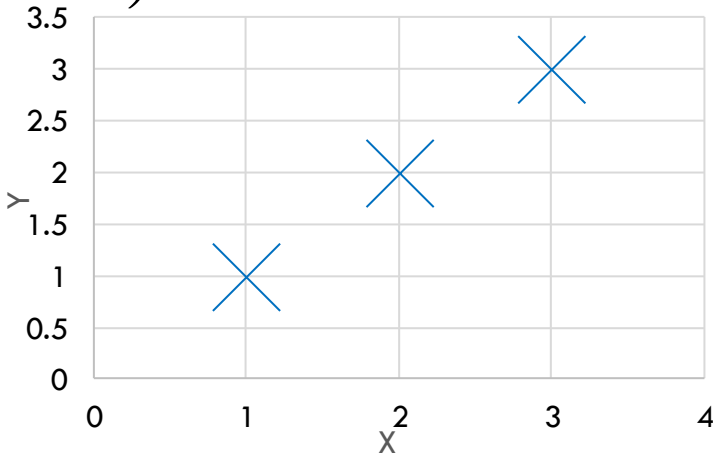
$$J(\theta) = \frac{1}{2n} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

x	y
1	1
2	2
3	3

Toy example

$\theta_0 = 0$ and $\theta_1 = 1$

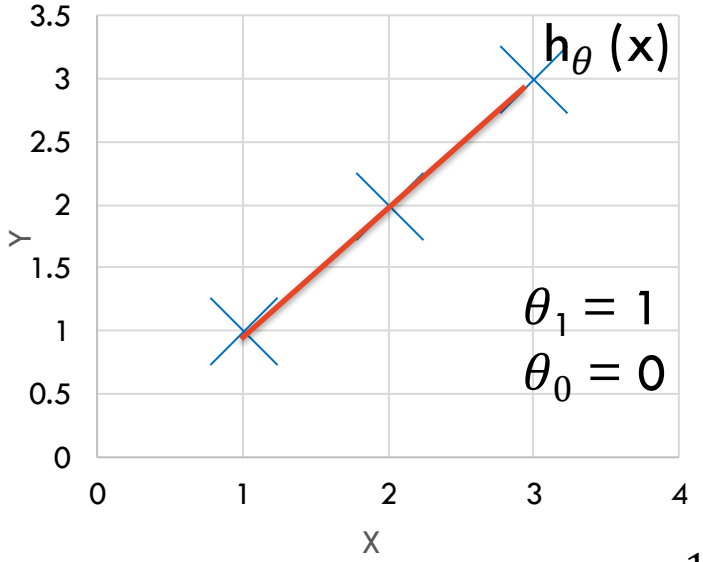
$Y = h_\theta(X) = X$

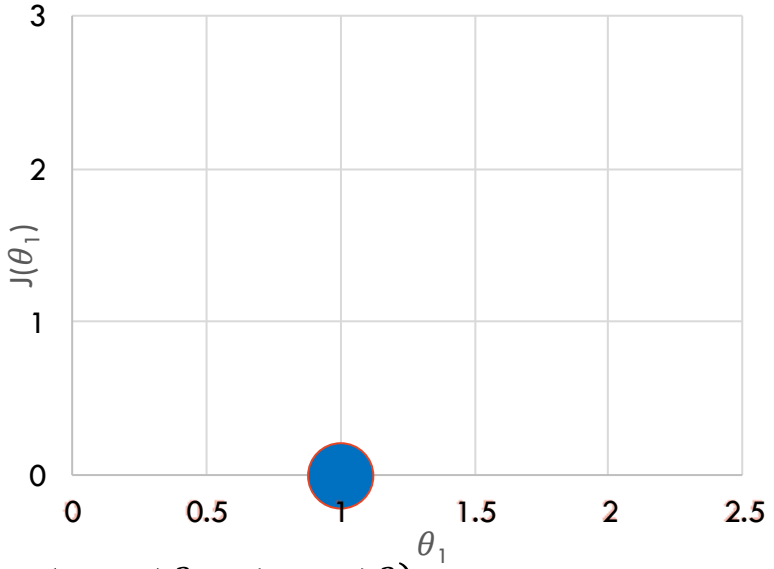


Intuition behind cost function (2 of 4)

We are only interested in how θ_1 change affects the error

$$J(\theta) = \frac{1}{2n}\sum_{i=1}^{n}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$$



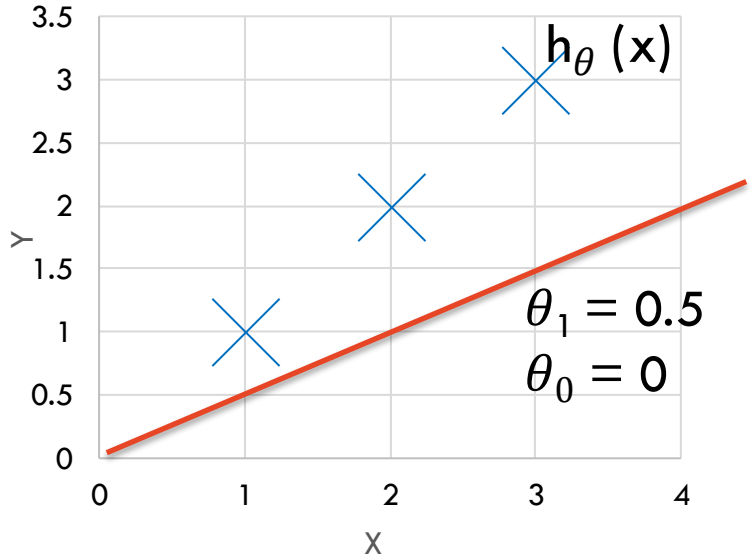


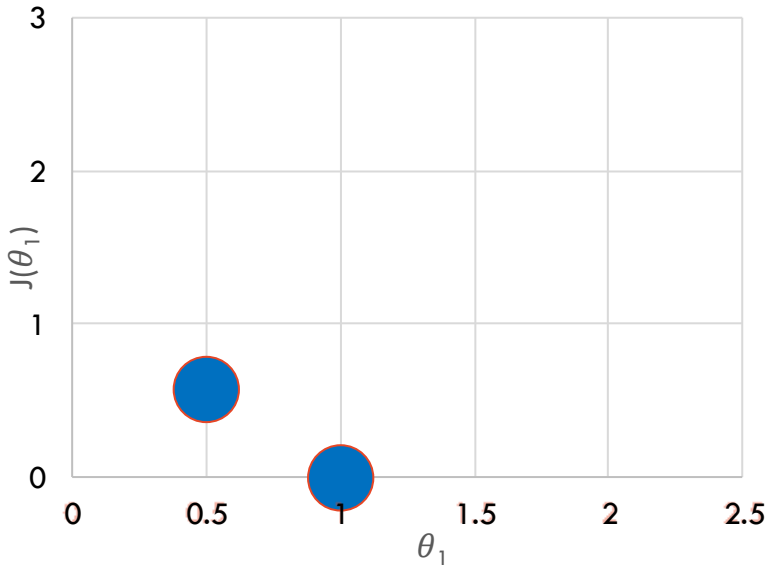
$$J([0,1]) = \frac{1}{2\times 3}\left((1-1)^2 + (2-2)^2 + (3-3)^2\right) = 0$$

Intuition behind cost function (3 of 4)

the value of θ_0 is fixed to 0, now we change the value of θ_1 to 0.5

The prediction function becomes $\hat{y} = 0.5x$



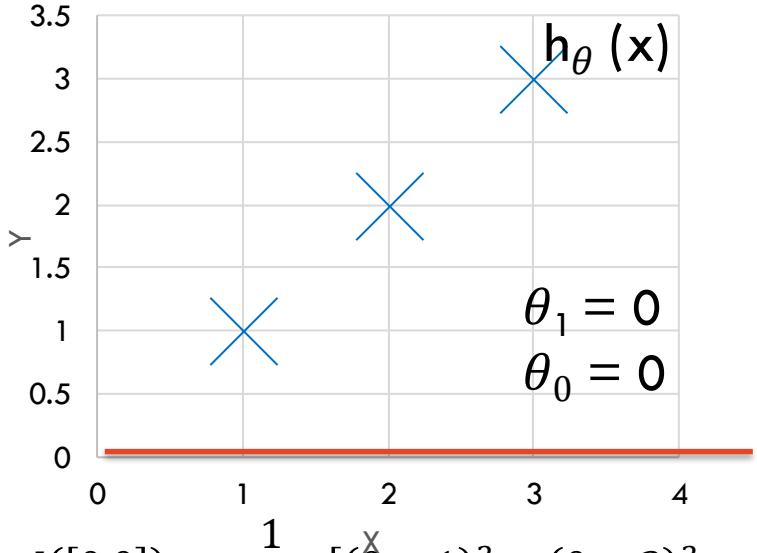


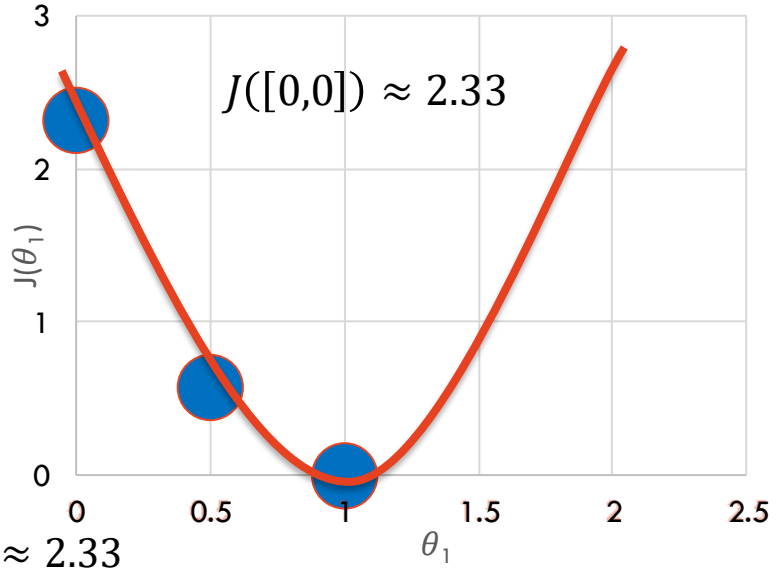
$$J([0,0.5]) = \frac{1}{2 \times 3}[(0.5-1)^2 + (1-2)^2 + (1.5-3)^2] \approx 0.58$$

Intuition behind cost function (4 of 4)

the value of θ_0 is fixed to 0, now we change the value of θ_1 to 0

The prediction function becomes $\hat{y} = 0x = 0$





$$J([0,0]) = \frac{1}{2 \times 3}[(0-1)^2 + (0-2)^2 + (0-3)^2] \approx 2.33$$

Has only one global minimum

Learning Means Finding the Lowest Cost

- Each value of θ_1 gives a different fitted line.
- Each fitted line gives a different cost $J(\theta_1)$.
- If we plot many parameter values against their cost, we get a cost curve.

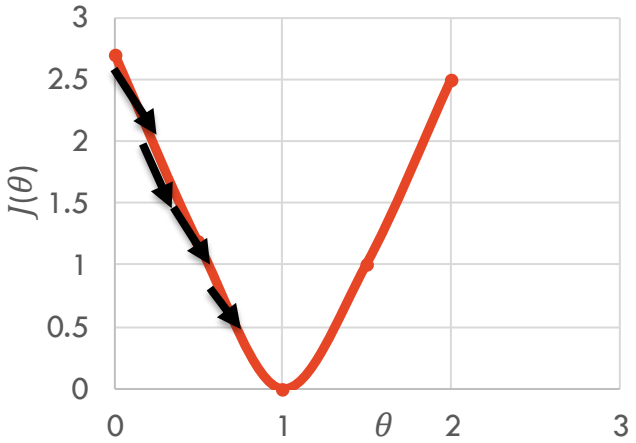
θ_1	Model	Cost	Interpretation
0	$\hat{y} = 0$	high	Too flat
0.5	$\hat{y} = 0.5x$	medium	closer
1	$\hat{y} = x$	lowest	Perfect for the toy data

Why gradient descent

- A model is trained by finding parameters that minimise the cost
- For OLS linear regression, this can be solved directly
- For many models, a direct solution is difficult or unavailable.
- Gradient descent finds good parameters by improving them step by step.

Gradient Descent

- Think of the cost function as a curve over parameter values.
- A point on the curve represents the current parameter value.
- The slope tells us which direction increases the cost.
- To reduce the cost, move in the opposite direction.

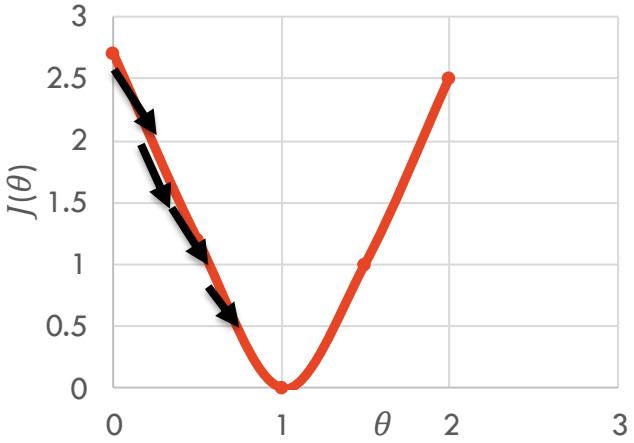


Gradient descent

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha * \frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta})$$

α is a learning rate (taking a small value)
e.g. $\alpha = 0.05$



$$\theta_j \leftarrow \theta_j - \alpha * \frac{1}{n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)} \right) x_j^{(i)}$$

simultaneous update for
$j = 0 \ldots d$ parameters/weights

Gradient descent always converges to the global minimum (assuming α is small)

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) &= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}) - y^{(i)} \right)^2 \\
&= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^{n} \left(\sum_{k=0}^{d} \theta_k \, x_k^{(i)} - y^{(i)} \right)^2 \\
&= \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{k=0}^{d} \theta_k \, x_k^{(i)} - y^{(i)} \right) \times \frac{\partial}{\partial \theta_j} \left(\sum_{k=0}^{d} \theta_k \, x_k^{(i)} - y^{(i)} \right) \\
&= \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{k=0}^{d} \theta_k \, x_k^{(i)} - y^{(i)} \right) x_j^{(i)} \\
&= \frac{1}{n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)} \right) x_j^{(i)}
\end{aligned}$$

Intuition behind gradient descent

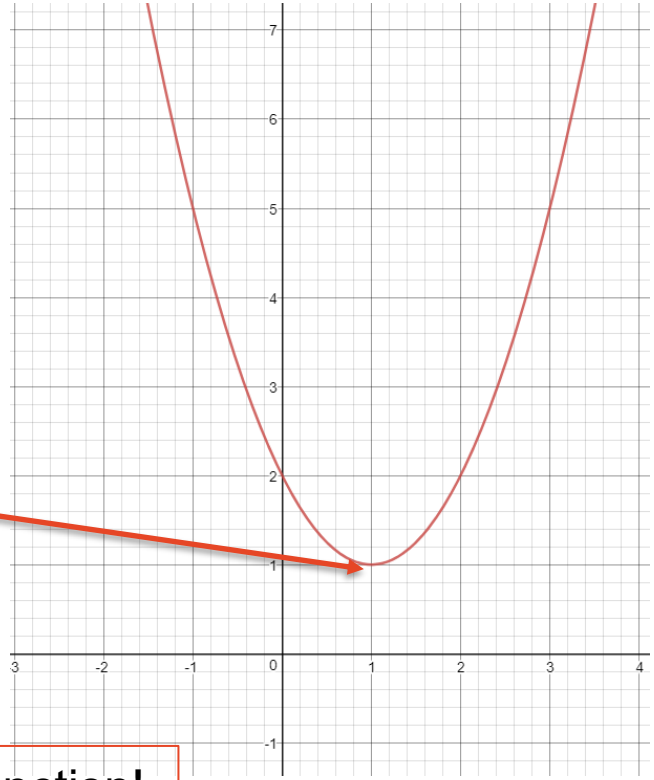
- Given simple cost function

$$J(w) = w^2 - 2w + 2,$$

find the value of w to minimise $J(w)$

- From Calculus, by finding the derivative and set it equal to zero:

$$\frac{dJ(w)}{dw} = 2w - 2 = 0 \Longrightarrow w = 1$$



But this is hard to solve for more complicated cost function!

Intuition behind gradient descent

- So, we pick a random number
  - Let w= 3, which obviously is wrong
  - Step 1: we take the derivative of the function
    - $\frac{dJ(w)}{dw} = 2w - 2$
  - Step 2: we study the derivative at the point we guessed $(x = 3)$
    - $\frac{dJ(w)}{dw} = 2 * 3 - 2 = 4$
  - Given that the derivative is positive, we know that the value is getting larger moving to the right (forward)
  - Therefore, we need to move to the left (go backword)

Intuition behind gradient descent

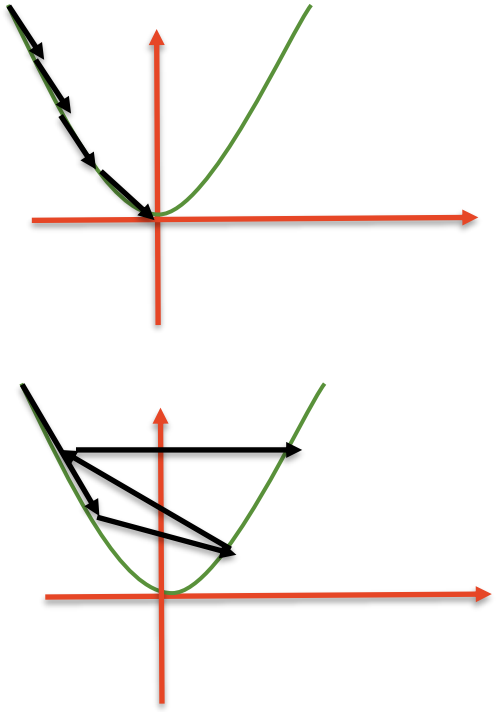
- If we have guessed $w = -1$ instead, the derivative would have been -4
  - then we would know that the function is getting smaller
- By studying the derivative of the current guess, we know if we are getting closer or further away from the minimum
- So here is the equation
  - $w_{i+1} = w_i - \alpha * \frac{dJ(w)}{dw}$, α is the learning rate, e.g. $\alpha = 0.2$
- Given our example, we guessed $w_0 = 3$
  - $w_{i+1} = w_i - 0.2 * \frac{dJ(w)}{dw}$
  - $w_1 = 3 - 0.2 * 4 = 2.2$

Intuition behind gradient descent

- We repeat this process again at $w_1 = 2.2$
  - $\frac{dJ(w)}{dw} = 2w - 2$
  - $\frac{dJ(w=2.2)}{dw} = 2 * 2.2 - 2 = 2.4$
  - $w_2 = 2.2 - 0.2 * 2.4 = 1.72$, we moved closer
- At $w_2 = 1.72$
  - $\frac{dJ(w)}{dw} = 2w - 2$
  - $\frac{dJ(w=1.72)}{dw} = 2 * 1.72 - 2 = 1.44$
  - $w_3 = 1.72 - 0.2 * 1.44 = 1.432$, we moved closer
- If we keep repeating this process, we can find the minimum point of the solution.

Selecting learning rate

- If α is small, gradient descent can be slow
- If α is too large, gradient descent might overshoot the minimum
- One good strategy is gradually shrinking the learning rate



How Much Data Is Used Per Update?

- Gradient descent updates the model parameters repeatedly
- The difference between batch, stochastic and mini-batch gradient descent is how much data is used to calculate each update

Method	Data used per update	Update behaviour
Batch	All training data	Stable but can be slow
Stochastic	One data point	Fast but noisy
Mini-batch	A small batch of data	Common compromise

All three methods learn one shared set of model parameters. They differ only in how the gradient is estimated at each step.

Batch vs stochastic gradient descents

- Batch: all n training examples are used to calculate one update
  - Repeat until converge $\left\{\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right) x_j^{(i)}\right\}$ for every j
  - Accurate but slow:
    - has to scan through the entire training set before taking a single step
    - costly operation if n is large
- Stochastic:

  For $i = 1$ to n

  $\left\{\theta_j \leftarrow \theta_j - \alpha \left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right) - y^{(i)}\right) x_j^{(i)}\right\}$, for every j
  - Fast, start making progress right away.
  - It may not converge to the minimum
- A trade-off between these two extremes is mini-batch

Test your understanding

- During training, the parameter updates frequently change direction: one update moves the parameters one way, and the next update moves them partly back. Which training process is most likely to show this behaviour?
  - A. Batch gradient descent
    B. Stochastic gradient descent
    C. Mini-batch gradient descent
    D. Closed-form ordinary least squares

Logistic Regression



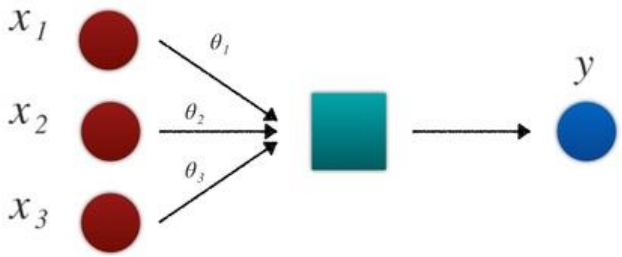
THE UNIVERSITY OF SYDNEY

Regression vs classification

- Regression predicts a numerical value
- Output is a continuous variable (real value)
- E.g., predict house price

- Classification predicts a class to each example
- Output is a discrete / categorical variable
- E.g., predict whether a customer will default or not

Logistic regression



- Predict probability of categorical label
- E.g., probability of defaulting on a loan given
  - Amount of debt
  - Late payment count

http://www.toshistats.net/101-4-logistic-regression/

Logistic regression for classification

- In binary classification, the target is usually: $Y \in \{0,1\}$
  - Y=1: positive class
  - Y=0: negative
- A linear regression model can predict any real number
  - $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_1 x_1 + .. + \theta_d x_d$
- For classification, we often want
  - $\hat{p} = P(Y = 1|X)$

Logistic Regression Model

- Step 1: calculate a linear score:

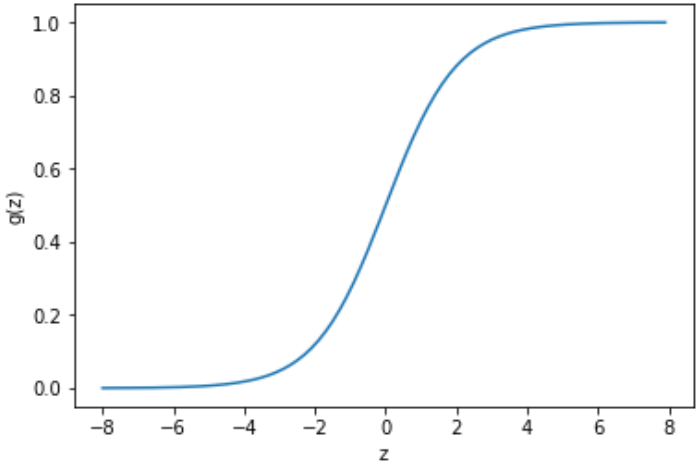
$$z = \theta_0 + \theta_1 x_1 + \theta_1 x_1 + .. + \theta_d x_d$$

- Step 2: convert the score into a probability:

$$\hat{p} = P(Y = 1|X) = \frac{1}{1 + e^{-z}}$$

- The sigmoid function keeps the output between 0 and 1

$$\lim_{z \to \infty} \frac{1}{1 + e^{-z}} \to 1$$



$$\lim_{z \to -\infty} \frac{1}{1 + e^{-z}} \to 0$$

From probability to class label

- Logistic regression predicts:

$$\hat{p} = P(Y - 1|X)$$

- A threshold converts probability into a class prediction:

$$\hat{Y} = \begin{cases} 1, \hat{p} \geq 0.5 \\ 0, \hat{p} < 0.5 \end{cases}$$

- The threshold can be adjusted depending on the problem

Training Logistic Regression

- We need a cost function that rewards high probability for the correct class.
- Logistic regression commonly uses log loss / cross-entropy, instead of Least Square error
- The cost is high when the model is confidently wrong
- The formula is

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)]$$

Cross entry loss example

- The individual case loss: $-[y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)]$

Actual class	$\hat{p} = P(Y = 1\|X)$	Interpretation	Cross-entropy loss
1	0.95	Confident and correct	$-\log(0.95) = 0.05$
1	0.60	Correct, but less confident	$-\log(0.60) = 0.51$
1	0.10	Confident but wrong	$-\log(0.10) = 2.30$
0	0.05	Confident and correct	$-\log(0.95) = 0.05$
0	0.40	Correct but less confident	$-\log(0.60) = 0.51$
0	0.90	Confident but wrong	$-\log(0.10) = 2.30$

Review

Tips and tricks

- Compare ML models to the simplest baseline first; Iterate
- Best strategy is often a simpler model with more/better data
- Always test on held-out data that hasn't been used for training

Tips and tricks

- Make sure features are on a similar scale.
- Rescale features to have zero mean and unit variance
  - Let $\bar{x}_j$ be the mean of feature j: $\bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_j^{(i)}$
  - Replace each value with:

$$x_j^{(i)} = \frac{x_j^{(i)} - \bar{x}_j}{s_j}$$

  - s_j is the standard deviation of feature j
- After recalling features, it is easier to find a learning rate that works well for all weights.
  - If the features are on vastly different scales, a learning rate that works well for updating one weight might be too large or too small to update the other weight equally well.

W9 review: Linear regression & logistic regression

Objective

Learn techniques for supervised machine learning, with tools in Python.

Lecture

- Simple linear regression
- Multiple linear regression
- Gradient descent
- Logistic regression

Readings

- Data Science from Scratch, Ch. 8, 14, 15, 16

Exercises

- sklearn: regression